

Github link

[Link to github](#)

Running the application

Requires:

- Atleast Java 25

Navigate to an safe directory and run the following code to get the code, and to navigate into the project:

```
git clone https://github.com/HeyItsAp/assignment4ALGDATA
cd assignment4ALGDATA
```

Run the following command to run at test the current codebase:

```
java -jar out/artifacts/assignment4ALGDATA_jar/assignment4ALGDATA.jar
```

If it does not work you can try through packaging on your own using **maven** and the attached **pom.xml**

OR read the following explation of the code for assesment:

The code

Assume each class has a appropriate getters and setters-methods.

Part 1: Using a double linked list to add long numbers

To make a linked list a node needs a pointer to the next and previous, along with its value, which is represented in `numberNode.java`:

```
public class numberNode {
    private numberNode neste;
    private numberNode forrige;
    private int value;

    public numberNode(numberNode neste, numberNode forrige, int value){
        this.neste = neste;
        this.value = value;
        this.forrige = forrige;
    }

    public numberNode(int value) {
        this.neste = null;
        this.forrige = null;
        this.value = value;
    }
}
```

For proper wrapper class around the `numberNode` or to actually make a linked list, we make a dedicated class `numberList.java`. This class also contains methods to print out and add nodes to the list:

```
public class numberList{
    private numberNode hode; // As in ###X not X###.
    private numberNode hale;

    public numberList(){
        this.hode = null;
        this.hale = null;
    }

    public void getFullList(){
        numberNode node = hode;
        while (node != null) {
            System.out.print(node.getValue() + " <-> ");
            node = node.getNeste();
        }
    }

    public void add(int value){
        numberNode nyNode = new numberNode(value);

        if (hode == null) {
            hode = nyNode;
            hale = nyNode;
        } else {
            nyNode.setForrige(hale);
            hale.setNeste(nyNode);
        }
    }
}
```

```

        hale = nyNode;
    }
}

```

To make it possible to create a number with too many digits allowed, we use a string to iterate and add each digit into a new Linked list:

```

public static numberList fromString(String digits) {
    numberList list = new numberList();
    for (char c : digits.toCharArray()) {
        if (!Character.isDigit(c)) {
            throw new IllegalArgumentException("Not a digit: " + c);
        }
        list.add(c - '0');
    }
    return list;
}

```

Following code is the adding algorithm for adding two lists together. It uses a helper function to add a integer to the left, used when actually adding the rest. Algorithm start from the right and adds each digits together and sends the rest. It does this until there isnt two digits to add and no rest.

```

public void addFirst(int value) { // add at the head
    numberNode nyNode = new numberNode(value);
    if (hode == null) {
        hode = hale = nyNode;
    } else {
        nyNode.setNeste(hode);
        hode.setForrige(nyNode);
        hode = nyNode;
    }
}

public numberList additionWithAnotherList(numberList otherList){
    numberList result = new numberList();
    numberNode haleOriginal = hale;
    numberNode haleOther = otherList.getTail();
    int rest = 0;

    while( haleOriginal != null || haleOther != null || rest > 0){
        int sum = rest;
        if (haleOriginal != null) { sum += haleOriginal.getValue(); haleOriginal =
haleOriginal.getForrige(); }
        if (haleOther != null) { sum += haleOther.getValue(); haleOther =
haleOther.getForrige(); }
        result.addFirst(sum % 10);
        rest = sum / 10;
    }
}

```

```
        return result;
    }
}
```

Part 2: Adding words to a Binary Search tree.

A node should contain a potential left and/or right children along with its string-value. `numberNode.java` represents this implementation:

```
public class treNode {
    private treNode left;
    private treNode right;
    private String value;

    public treNode(String value) {
        this.left = null;
        this.right = null;
        this.value = value;
    }
}
```

A wrapper class `tre.java` utilizes the `treNode.java` class and has methods for insertion and printing it out.

```
public class tre {
    private treNode root;

    public tre() {
        this.root = null;
    }
}
```

These two methods starts a recursion call to add a new value according to the properties of Binary Search tree. The sorting algorithm is alphabetical. For example; A root with bb, will have 'aa' to the left and 'cc' to the right, then 'ab' will be the right node of 'aa':

```
public void insert(String value){
    root = insertRecusively(root,value);
}

static treNode insertRecusively(treNode node, String value){
```

```

    if (node == null) return new treNode(value);

    if (value.compareTo(node.getValue()) > 0){ // if a less than b, left node
        node.setRight(insertRecusively(node.getRight(),value));
    } else if (value.compareTo(node.getValue()) < 0){ // if a comes after b, right
node
        node.setLeft(insertRecusively(node.getLeft(), value));
    }
    return node;
}

```

toString() method is overwritten and uses a simple level-based print out method. The rot will start to the left work to its right. For example, a input of 'bb aa cc ab', will printed out as:

```

    cc
bb
    ab
    aa

```

('aa' right node is 'ab')

```

public String toString(){
    StringBuilder sb = new StringBuilder();
    print(root, 0, sb);
    return sb.toString();
}
private void print(treNode node, int depth, StringBuilder sb) {
    if (node == null) {
        return;
    }
    print(node.getRight(), depth + 1, sb);
    sb.append("    ".repeat(depth)).append(node.getValue()).append("\n");
    print(node.getLeft(), depth + 1, sb);
}

```